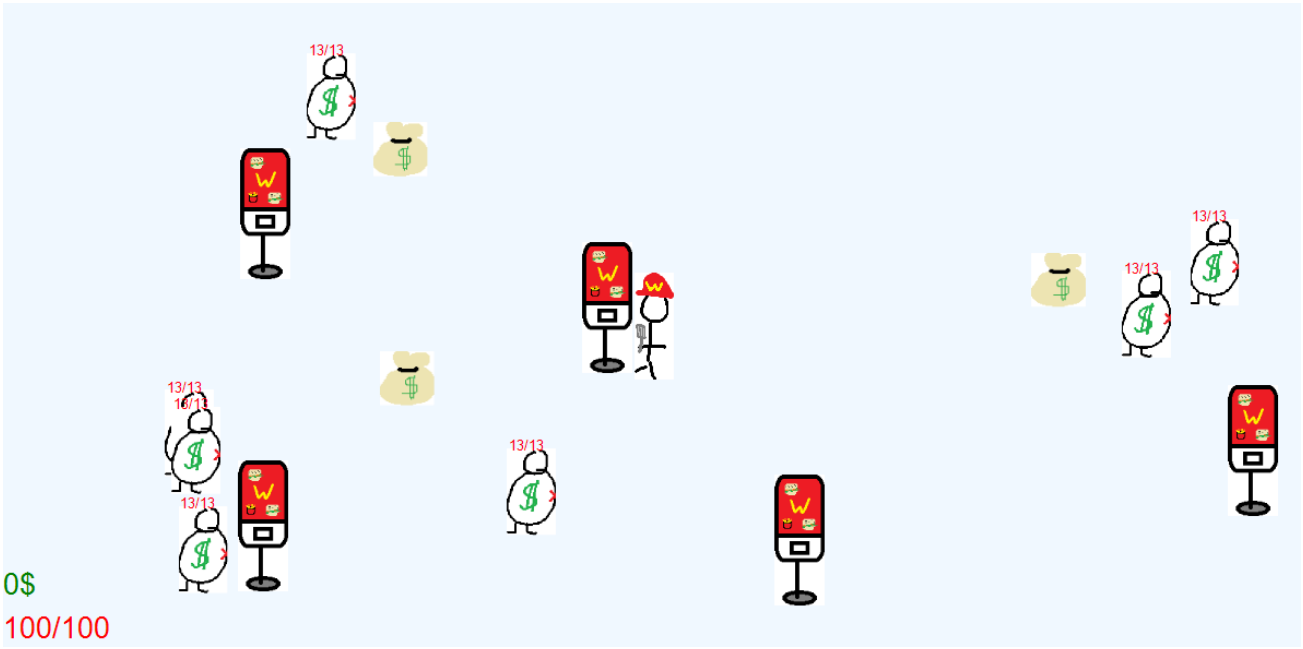
P_OO Shoot Me Up Rapport de Projet



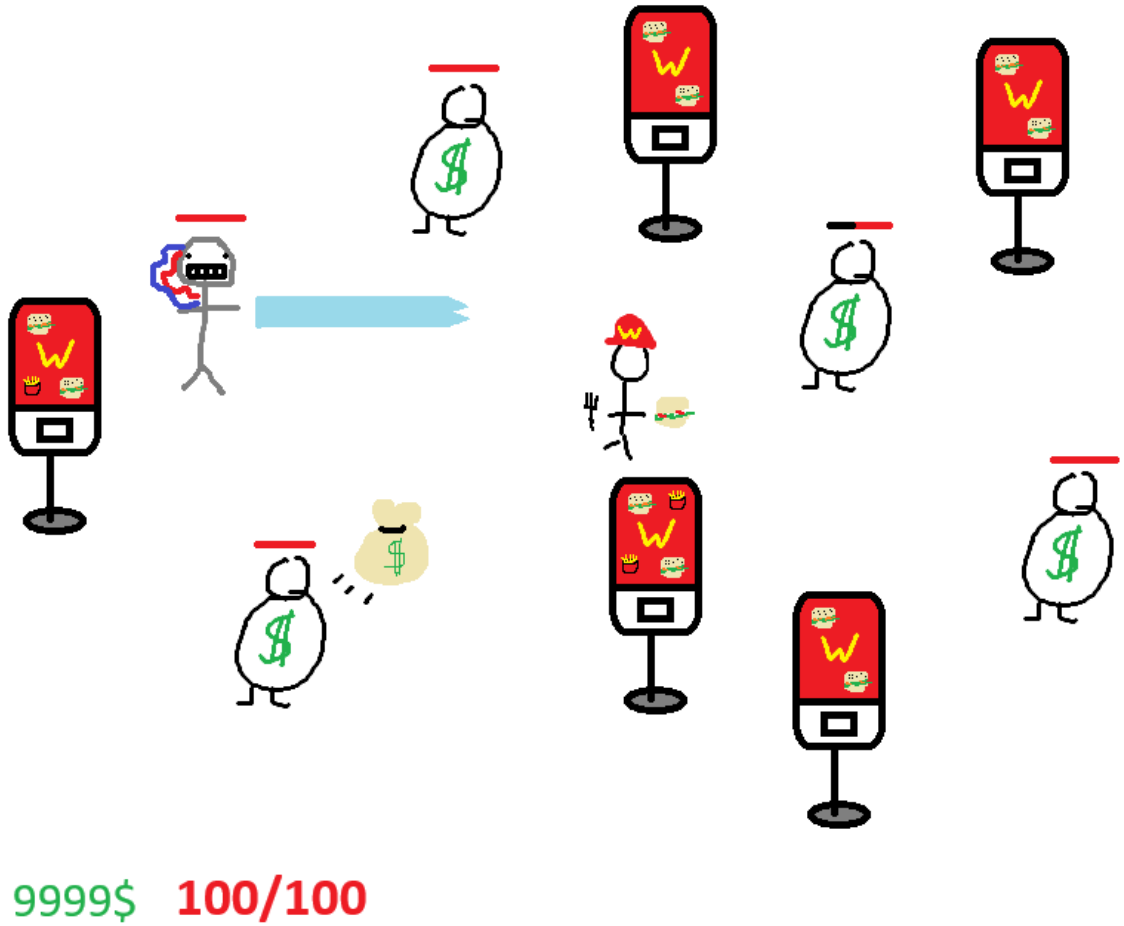
Antoni Débaz – MID2B – Avenue de Valmont 28

Table des matières

P_OO Shoot Me Up Rapport de Projet 1
Introduction 3
Objectifs produit 3
Objectifs pédagogiques 4
Analyse fonctionnelle 4
#1 — pouvoir faire bouger mon joueur avec WASD 4
#2 — pouvoir tirer des projectile sur mes enemies. 5
#3 — pouvoir faire des dégâts de corp à corp. 6
#4 — En temps que joueur je souhaite avoir des ennemis de type manager afin de pouvoir gagner de l'argent 6
#5 — avoir des ennemis de type robot 7
#6 — les ennemis de type robot peuvent me tirer dessus 8
#7 — les ennemis de type manager peuvent me tirer dessus 9
#8 — avoir une barre de vie 9
#9 — avoir une barre d'argent 10
#10 — avoir des obstacles afin de pouvoir me protéger des projectiles ennemies. 10
Planification Initiale 11
Rapport de tests 11
HelpersTest 11
ManagerTests 12
PlayerTests 12
RobotTests 13
MSTestSettings 13
Conception et diagramme de classe 14
Explications 14
Utilisation de l'IA 16
Bilan du déroulement du projet 17
Bilan produit 18
Conclusion 18

Introduction

Objectifs produit



Mon jeu portera sur le thème d'un employé de fast-food (Wacdonald's) devant affronter sa hiérarchie et les robots voulant le remplacer. Une partie type se passera comme ceci :

Ennemis :

Le jeu commencera avec peu d'ennemis de type manager. Ils attaqueront en lançant des sacs d'argent en direction du joueur et se déplaceront aléatoirement. Plus le joueur avancera dans le jeu plus ces ennemis deviendront puissant, ils auront de plus en plus de PV et leurs attaques feront de plus en plus de dégâts. Plus ces ennemis seront puissants plus l'argent récupéré en les tuant sera important.

Au bout d'un certain temps, des ennemis de type robots apparaitrons. Ces ennemis feront des attaques « faisceaux » qui ne pourront aller que à la verticale

ou à l'horizontal. Ils ne feront pas tomber d'argent en étant tué. Il se déplaceront pour essayer d'être en dessous du joueur est leur attaque aura un cooldown.

Joueur :

Le joueur pourra se déplacer avec les touches WASD et pourra lancer des burgers dans la direction de sa souris avec clic gauche. Il pourra aussi faire des dégâts de corp à corp avec la spatule et le clic droit.

Le joueur pourra également se cacher derrière les bornes wacdonald's pour se protéger.

Victoire/Défaite :

Le but du jeu est de gagner 1'000'000$ afin de devenir actionnaire majoritaire du fast food. Le joueur perd la partie si sa vie tombe à 0.

Objectifs pédagogiques

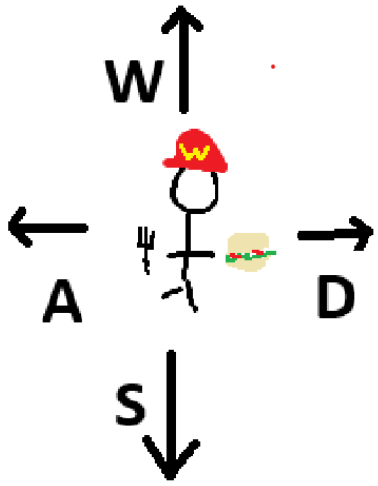
Nous allons faire ce projet pour développer nos compétences sur les points suivants :

- Amélioration de nos compétences en C# et programmation orienté objet en général.
- Rédaction de rapport
- Rédaction d'un journal de travail
- Gestion de ressources temporelles limitées
- Gestion de repository Github (release, issues)
- Planification de projet

Analyse fonctionnelle

#1 — pouvoir faire bouger mon joueur avec WASD

En tant que joueur, je veut pouvoir utiliser les touches WASD afin de pouvoir faire bouger mon joueur dans chaque direction.

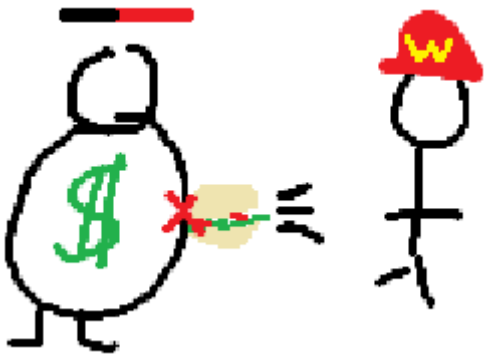


Critère d'acceptation :

- Le joueur va dans la direction correspondante tant que je maintient la touche
- Le joueur arrête d'avancer dès le moment ou le joueur arrête d'appuyer sur la touche
- Le joueur s'arrête lorsque il essaie d'aller dans les obstacles
- Le joueur s'arrête lorsqu'il arrive au bord de l'écran
- Si j'appuie sur deux touches, le joueurs va en diagonal

#2 — pouvoir tirer des projectile sur mes enemies.

En tant que joueur, je souhaite pouvoir utiliser la touche clic gauche afin de pouvoir tirer des projectile sur mes enemies.



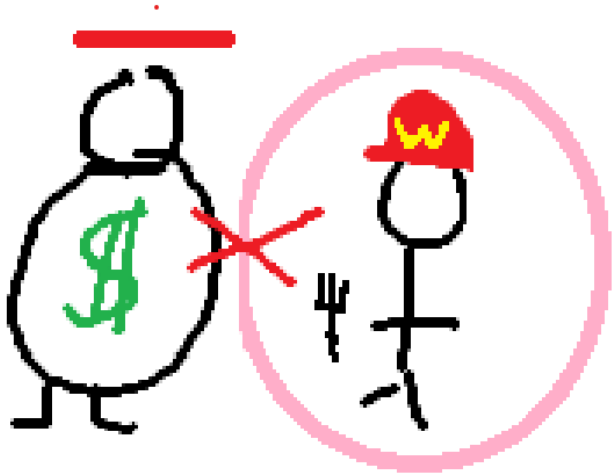
Critère d'acceptation :

- Lorsque je fais clic gauche, un projectile burger pars en direction de ma souris.

- Le projectile est détruit si il rencontre le bord de la carte
- Le projectile à un cooldown de 1sec après avoir été tiré.

#3 — pouvoir faire des dégâts de corp à corp.

En tant que joueur, je souhaite pouvoir utiliser la touche clic gauche afin de pouvoir tirer des projectile sur mes enemies.

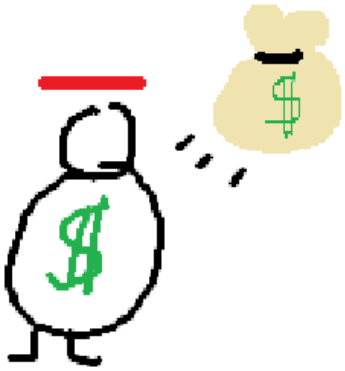


Critère d'acceptation :

- Lorsque je fais clic gauche, un projectile burger pars en direction de ma souris.
- Le projectile est détruit si il rencontre le bord de la carte
- Le projectile à un cooldown de 1sec après avoir été tiré.

#4 — En temps que joueur je souhaite avoir des ennemis de type manager afin de pouvoir gagner de l'argent

En tant que joueur je souhaite veut des ennemis de type manager afin de pouvoir gagner de l'argent

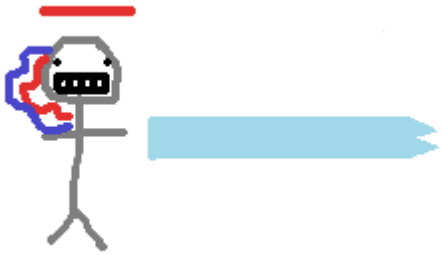


Critère d'acceptation

- Un manager doit pouvoir se deplacer aléatoirement sur la carte
- Un manager ne peut pas dépasser les limites de la carte
- Un manager possède 13 PV
- Les missiles burger disparaissent si ils touchent un manager
- Un manager prend 3PV de dégat quand touché par une attaque de corp à corp
- Un manager dans un rayons égal à la moitié de la largeur du joueur lors d'une attaque de mélée sera affécté
- Un manager prend 5PV de dégat quand touché par une attaque burger
- Un manager disparait lorsque ces points de vie sont inférieur ou égal à 0
- Les manager apparaissent sur les coté de la carte
- Il doit toujours y avoir 7 manager sur la carte

#5 — avoir des ennemis de type robot

En temps que joueur je souhaite avoir des ennemis de type robot afin de pouvoir gagner de l'argent



Critère d'acceptation

- Un robot se déplace sur la carte en essayant de toujours se positionner directement au dessus/dessous/à droite/ à gauche du joueur
- Un robot possède 21 PV
- Un robot de fais pas tomber d'argent
- Les robots ne commence à apparaitre que quand le joueur atteint 100'00$ (niveau 2) et leurs PV double quand il atteint 500'00$ (niveau 3).
- Les missiles burger disparaissent si ils touchent un robot
- Un robot prend 3PV de dégat quand touché par une attaque de corp à corp
- Un robot dans un rayons égal à la moitié de la largeur du joueur lors d'une attaque de mélée sera affécté
- Un robot prend 5PV de dégat quand touché par une attaque burger
- Un robot disparait lorsque ces points de vie sont inférieur ou égal à 0
- Dès que le joueur atteint le niveau 2, il doit toujours y avoir 4 robot sur la carte

#6 — les ennemis de type robot peuvent me tirer dessus

En tant que joueur je souhaite que les ennemis de type robot puisse me tirer dessus afin d'ajouter du challenge



Critère d'acceptation

- Un faisceau est tiré par un ennemi de type Robot
- Les faisceaux ne peuvent aller que dans les quatre direction cardinal (pas de diagonal)
- Les faisceaux vont dans la direction que le joueur avait quand le robot à commencer son animation de tire.

• Les faisceaux font 25PV de dégats au joueur peut importe le niveau du robot qui les a tiré.

• Les faisceaux disparaissent si ils touchent un obstacle.

• Un robot prends 3 sec à lancer une attaque faisceau lorsqu'il est en position. Il ne peut plus bouger pendant cette période

#7 — les ennemis de type manager peuvent me tirer dessus

En tant que joueur je veut que les ennemis de type manager puisse me tirer dessus afin d'ajouter du challenge



Critère d'acceptation

• Un sac doit être lancé par un ennemi de type manager

• Les sacs vont en directions de la position que le joueur avait quand le sacs à été lancé.

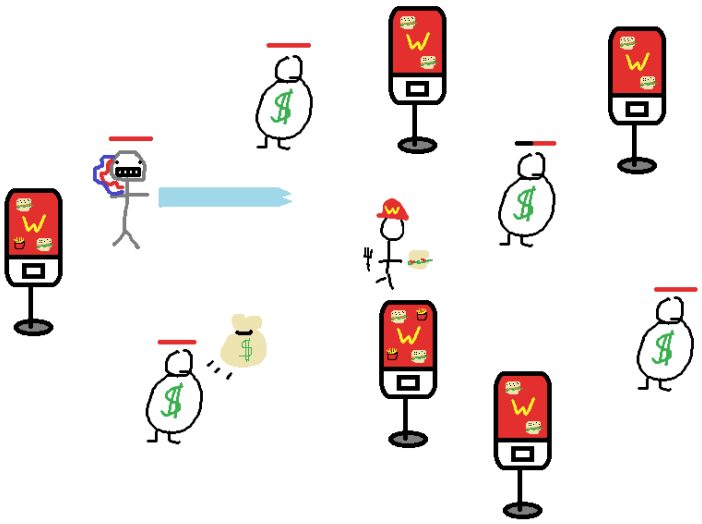
• Le sacs ne change pas de direction

• Le sac fait 10PV de dégat si il à été lancé par un ennemis manager de niveau 1, 15 PV si c'est un manager de niveau 2 et 20PV si c'est un manager de niveau 3.

• Un sac doit prendre 4 sec à être recharger par un ennemi de type manager

#8 — avoir une barre de vie

En tant que joueur je souhaite avoir une barre de vie afin de pouvoir perdre la partie

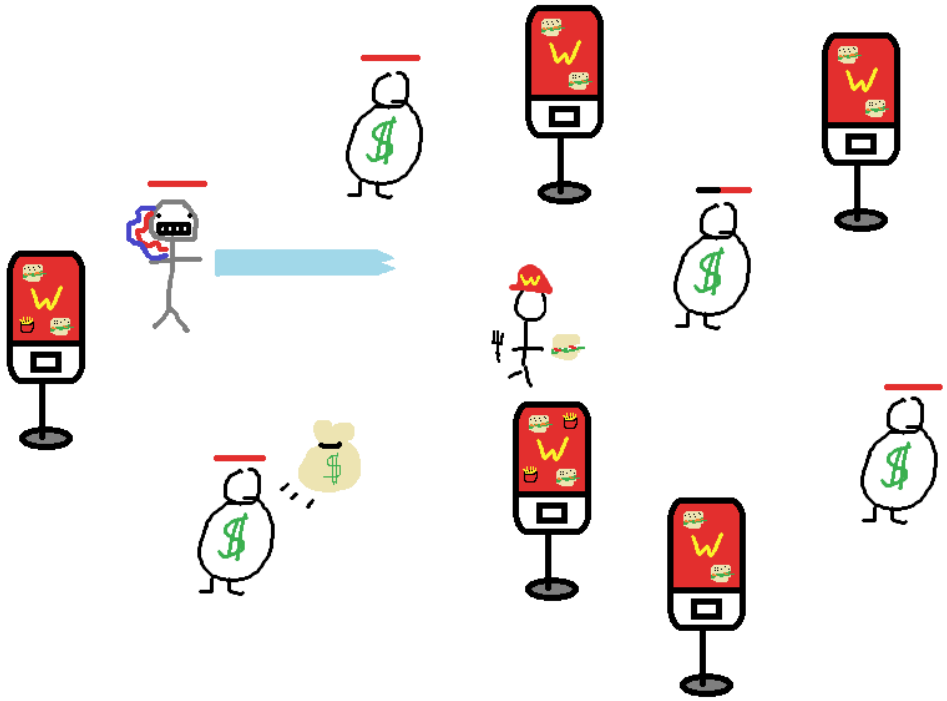


Critère d'acceptation

- Lorsque le nombre de point de vie atteint 0, l'écran de mort s'affiche
- La barre de vie est de 100/100 au démarrage
- La barre de vie se situt en bas à gauche

#9 — avoir une barre d'argent

En tant que joueur je souhaite avoir une barre d'argent afin de pouvoir gagner la partie



Critère d'acceptation

- Lorsque la barre d'argent atteint 1'000'000, l'écran de réussite s'affiche.
- Lorsque un manager meurt, le nombre d'argent du joueur augment de 5'000$
- L'argent que fait tomber le manager est quadruplé et ses PV sont doublé quand le joueur atteint 100'000$ (manager de niveau 2). Il le sont à nouveaux quand il atteint 500'000$ (manager de niveau 3).

#10 — avoir des obstacles afin de pouvoir me protéger des projectiles ennemies.

En tant que joueur je veut avoir des obstacles afin de pouvoir me protéger des projectiles ennemies.



Critère d'acceptation

- L'obstacle est indesctuctible
- Les ennemis et les projectils ne peuvent pas passer à travers les obstacles
- Les missiles burger et les faisceaux disparaissent si ils touchent un obstacle
- Un robot ou un manager doit trouver une nouvelle destination si il bute sur un obstacle

Planification Initiale

Date de fin prévu de la tâche	Tâche à faire (lien vers leurs User Stories)
18.09.2026	Ajout du mouvement WASD
18.09.2026	pouvoir tirer des projectile sur mes enemies
25.09.2026	pouvoir faire des degats au corp à corp
25.09.2026	Ajouter un compteur de PV
25.09.2026	Ajouter un compteur d'argent
02.10.2026	Créer des ennemies de type manager
02.10.2026	Créer les projectiles des managers
09.10.2026	Créer des ennemies de type robot
30.10.2026	Créer les projectiles des robots
30.10.2026	Créer les obstacles

Rapport de tests

Voici une explication des différents tests existant dans mon projet :

HelpersTest

Cette classe contient deux tests :

TestDistance **:** Vérifie que le helper « MathHelper.Distance » fonctionne en lui donnant deux points ayant des coordonnées chacune et en la comparant avec la valeur qu'elle devrait avoir. Le premier point vaut (0,0) et le second à les

coordonné (3,4) avec un simple théorème de Pythagore on calcul que leur distance doit être de 5 et on compare ça avec la sortie du helpers.

TestAreTouching **:** Vérifie que le helpers « MathHelper.AreTouching » fonctionne en lui donnant deux rectangles ayant chacun des coordonnées, une largeur et une hauteur. On regarde ensuite si le helper arrive à correctement prédire que les rectangles se touchent. Le premier rectangle a une position (0,0), une hauteur et une largeur de 15 et le deuxième à la position (15,15) ainsi qu'une hauteur et une largeur de 20. Le test doit ensuite regarder que le helper voit bien que ses points se touchent.

ManagerTests

Cette classe contient deux tests et utilise une methode Setup lancé par [TestInitialize] afin de réinitialiser le champ de batail entre les tests :

ManagerFires **:** Cette méthode instancie un manager ainsi qu'un champ de bataille. Elle supprime ensuite tous les obstacles présents dans le champ de bataille. Puis elle attend le nombre de frame que le Manager prend à charger avant de vérifier si une attaque de type sac d'argent a été ajouté dans le champ de bataille.

ManagerMoves **:** Cette méthode instancie un manager puis attend deux frames. Il regarde ensuite si le Manager à changer de position.

ManagerStuckByObstacles : Cette méthode instancie un nouveau champ de bataille puis un nouveau manager et créée 4 obstacles qui l'entourent. Il laisse ensuite une frame passer puis regarde si la position du Manager à changer.

PlayerTests

Cette classe contient 7 tests et utilise une methode Setup lancé par [TestInitialize] afin de réinitialiser le champ de batail entre les tests :

PlayerBurgerAttack: Cette méthode créer un joueur puis simule un clic gauche. Elle regarde ensuite le nombre de balle présente dans le champ de bataille.

PlayerMeleeAtack : Cette méthode créer un joueur puis attends une frame le temps que le cooldown de départ passe puis simule un clic droit. Elle laisse encore une frame passer puis elle regarde si le joueur est en mode Melee ou non.

PlayerGoUp/PlayerGoDown/PlayerGoLeft/PlayerGoRight : Ces quatres méthodes instancie toutes un joueur puis simule le pressage de la touche pour aller dans leur direction respective (WASD). Elle laisse toutes une frame passer avant de regarder si la position du joueur à changer de manière égale à sa vitesse et dans la bonne direction.

PlayerCollisionWithObstacle : Cette méthode créer un joueur puis ajoute un obstacle dans le champ de bataille juste en face du joueur. Elle simule ensuite le fait de cliquer sur « W » pour avancer vers l'obstacle puis regarde si la position du joueur à changer. Le but est de voir si la collision fonctionne. Si oui, le joueur ne changera pas de position.

PlayerCollisionWithObstacle : Cette méthode créer un joueur au bord haut de la map. Elle simule ensuite le fait de cliquer sur « W » pour avancer vers le bord puis regarde si la position du joueur à changer. Le but est de voir si la collision fonctionne. Si oui, le joueur ne changera pas de position.

RobotTests

Cette classe contient deux tests et utilise une methode Setup lancé par [TestInitialize] afin de réinitialiser le champ de batail entre les tests :

RobotFires : Cette méthode instancie un nouveau champ de bataille puis un nouveau Robot. Elle supprime ensuite tous les obstacles du champ de bataille et attend que le Robot se mette en position et tire. Si le robot prend plus de temps à tirer que la taille de la map, alors c'est que le test ne fonctionne pas. Puis on regarde combien de tir sont présent dans le champ de batail pour valider ou non le test.

RobotMoves : Cette méthode instancie un nouveau champ de bataille puis un nouveau robot et supprime aussi tous les obstacles. Il laisse ensuite une frame passé puis regarde si la position du Robot à changer.

RobotStuckByObstacles : Cette méthode instancie un nouveau champ de bataille puis un nouveau robot et créée 4 obstacles qui l'entourent. Il laisse ensuite une frame passé puis regarde si la position du Robot à changer.

ProjectileTests

Cette classe contient 3 tests pour les 3 type de projectiles

MoneyBagDeplacementTest/FaisceauDeplacementTest/BurgerDeplacementTest : Pour chacun de ces tests, on initialise un objet de chaque type essayant d'aller vers 0,0 (ou le nord dans le cas du faisceaux). On laisse ensuite passer une Update puis on regarde si la position à changer.

MSTestSettings

J'ai du modifié le fichier MSTestSettings.cs et changer la première ligne en [assembly: DoNotParallelize]. Cela était nécessaire car le champ de batail est une ressource partagé entre tous les tests ce qui amènerais autrement des tests à écrire dans la classe « BattleMap » au moment ou un autre test allait le lire. J'ai donc du désactiver le parallélisme.

Conception et diagramme de classe



Explications

BattleMap

Dans le programme, la première classe à être instancié est le BattleMap. Il à comme argument le joueur. BattleMap met tous les autres objets à jour via Update en appelant le « Update » de chaque objet puis il les fait afficher en

appelant le « Render » de chaque objet (Update et Render sont appelé par NewFrame). Il detecte aussi les touche de clavier et les stockent dans une liste présente sur le joueur et lui transmet également les touches de la souris.

Joueur et ProjectileBurger

Une fois le Joueur instancié. Il peut se déplacer grâce à sa liste de touche et bûte sur les bords de la map (BattleMap stocke ses dimensions) ou les objets de type Obstacle. Il peut aussi se mettre en mode mêlée quand il reçoit un clic souri. Il a également une santé qui baisse lorsqu'il touche un objet de type Faisceau ou MoneyBag et une somme d'argent qui augmente lorsqu'un objet de type Manager meurt.

Le joueur peut instancier des projectiles Burger lorsqu'il reçoit un clic souri qui sont ensuite gérer par BattleMap en les stockant dans la liste allBurgers. Les burgers peuvent se déplacer sur la map

Les attaque mêlée et Burger ont tout les deux un temps de rechargement.

BattleMap de son côté instancie plusieurs listes d'éléments dont plusieurs Obstacle, plusieurs Manager et plusieurs Robot. Chaque liste à un nombre max d'élément gérer par leur constante respective.

Manager et MoneyBag

Les Managers se déplace aléatoirement sur la map en trouvant une nouvelle destination lorsqu'il butte sur un mur ou arrive à leur destination. Ils ont également une santé qui baisse quand ils sont trop proche d'un joueur en mode mêlée (La distance max est stockée dans le joueur). Il possède également un « loot » qui correspond à l'argent qu'ils font tomber à leur mort. Ils peuvent également être de niveau différent. Lesquels dépend de l'argent du joueur. Le threshold est définis par BattleMap.

Ils peuvent instancier des éléments de type « Sac d'argent » chaque fois que le nombre de temps définis par (FIRE_RATE) s'écoule, les managers tirent. Les sacs sont ensuite placés dans leur liste dans BattleMap. Les sacs d'argent vont en direction que le joueur avait lorsqu'ils ont été instanciés. Ils se détruise lorsqu'il arrive au bord de la map, qu'ils butent sur un obstacle ou qu'il touche le joueur.

Robot et Faisceau

Les Robot se déplace sur la map en trouvant une nouvelle destination qui doit toujours être directement au-dessus/dessous/droite/gauche du joueur. Il trouve un nouveau point ou aller si ils butent sur un obstacle. Ils ont également une santé qui baisse quand ils sont trop proche d'un joueur en mode mêlée (La distance max est stockée dans le joueur). Ils peuvent également être de niveau

différent. Lesquels dépend de l'argent du joueur. Le threshold est définis par BattleMap.

Ils peuvent instancier des éléments de type « Faisceau » chaque certain temps (FIRE_RATE). Les faisceaux sont ensuite placés dans leur liste dans BattleMap. Les faisceaux ne peuvent aller que dans les directions orthogonales ils ont donc une direction. Ils se détruise lorsqu'il arrive au bord de la map, qu'ils butent sur un obstacle ou qu'il touche le joueur.

Utilisation de l'IA

Voici une explication de tout mes chat avec l'IA :

Chat numéro 1 :

Historique :

https://claude.ai/share/2bf90d2b-864b-42cc-8b37-eaf1b68bf1c2

Contexte :

Mes tests ne voulaient pas se lancer et avais des erreurs étranges. J'ai donc demandé de l'aide à Claude qui m'a expliqué que mon MSTest tournait sur « .NET Framework 4.7.2 » à la place de « net8.0-windows7.0 ». J'ai donc recréé ma solution avec la bonne version de MSTest et mes tests on alors recommencer à fonctionner correctement.

Chat numéro 2:

Historique :

https://claude.ai/share/811b5089-de97-4356-8ea7-5ff4a40ace65

https://claude.ai/share/b1e4f4ad-725a-4019-8b77-c83d20ff485d

Contexte:

J'essayais de pinpoint la raison d'un bug dans mes tests. Pour une raison que je ne comprenais pas, certains tests était valide lors d'une execution et arrêtait de l'être ensuite. Claude m'a expliqué que la raison était que tous mes tests essayaient de lire et d'écrire dans la classe static BattleMap au même moment. La solution a été de désactiver le parallélisme des tests en allant dans le fichier MSTestSettings.cs et en modifiant la 1ère ligne en [assembly: DoNotParallelize]. Il a aussi fallu ajouter une méthode [TestInitialize] pour faire table rase du test précédent (vider les listes).

Chat numéro 3 :

Historique :

https://claude.ai/share/10e2149d-0df4-480c-a93a-7e6df141afcd

Contexte :

Lors de la création du diagramme de classe, j'ai fait le premier à la main puis j'ai donner tout mon code à claude et mon 1er diagramme comme model. J'ai ensuite copier-coller ses réponses et ai retirer les éléments superflus comme les fonctionnalité interne par exemple. J'ai aussi mis les différentes liaisons entre les diagrammes moi-même.

Bilan du déroulement du projet

Critique de mon planning

Date de fin prévu de la tâche	Date de fin réel de la tâche	Tâche à faire (lien vers leurs User Stories)
18.09.2026	18.09.2026	Ajout du mouvement WASD
18.09.2026	25.09.2026	pouvoir tirer des projectile sur mes enemies
25.09.2026	25.09.2026	pouvoir faire des degats au corp à corp
25.09.2026	02.10.2026	Ajouter un compteur de PV
25.09.2026	02.10.2026	Ajouter un compteur d'argent
02.10.2026	02.10.2026	Créer des ennemies de type manager
02.10.2026	02.10.2026	Créer les projectiles des managers
09.10.2026	09.10.2026	Créer des ennemies de type robot
30.10.2026	09.10.2026	Créer les projectiles des robots
30.10.2026	23.10.2026	Créer les obstacles

Comme vous pouvez le voir, toutes les tâches ont été compléter avec une marge d'une semaine par rapport à ce qui était prévu.

Problème rencontré

- Les ennemis de type manager ont été ajoutée avant le compteur de PV et le compteur d'argent contrairement au planning. J'ai réalisé que c'était plus logique car il fallait bien quelque chose qui me tirait dessus pour pouvoir perdre des PV.
- J'ai eu plusieurs problèmes avec les tests dû à la version du frameWork et au parallélisme comme expliquer au chapitre « Utilisation de l'IA -> Chat numéro 1 » et « Utilisation de l'IA -> Chat numéro 2 ».
- J'ai eu des problèmes avec la détection des touches du clavier et de la souris. J'ai cherché un peu partout dans le projet avant de trouver le fichier BattleMap.Designer.cs contenant KeyUp et KeyDown.
- J'ai eu des problèmes avec le fait de faire tourner les faisceaux des robots. Un collègue m'est venu en aide en me montrant un technique consistant à faire

tourner non pas l'objet mais le monde autour de lui pour donner l'impression qu'il était tourné.

Bilan produit

Finis ou non	Tâche à faire (lien vers leurs User Stories)
OUI	Ajout du mouvement WASD
OUI	pouvoir tirer des projectile sur mes enemies
OUI	pouvoir faire des degats au corp à corp
OUI	Ajouter un compteur de PV
OUI	Ajouter un compteur d'argent
OUI	Créer des ennemies de type manager
OUI	Créer les projectiles des managers
OUI	Créer des ennemies de type robot
OUI	Créer les projectiles des robots
OUI	Créer les obstacles

Comme vous pouvez le voir, tous ce qui était demander dans les US à été fait et la version finale de contient aucun bug majeur,

Conclusion personnelle

J'ai beaucoup apprécié ce projet car le C# et la programmation en général m'intéresse beaucoup. J'ai le sentiment d'avoir beaucoup progressé dans ce domaine. J'ai beaucoup aimé apprendre à propos des objets qui est un élément central de la programmation orienté objet. Je pense que cela va m'être très utile dans mes projets personnels à la maison.

Je pense aussi que le fait de s'entrainer à faire des tests est une bonne pratique car l'IA va nécessiter que de plus en plus de développeur soit à l'aise d'en écrire un grand nombre.